



Escola Politécnica de Enxeñaría de Ferrol

UNIVERSIDADE DA CORUÑA



Introducción al Aprendizaje Automático CURSO 2025/2026

Preparación de los datos y clasificación supervisada

M. U. en Ingeniería Industrial

AUTORES: ÁLVARO NAVEIRA CAAVEIRO
FRANCISCO RODRÍGUEZ GAVÍN

TUTORES: ALMA MARÍA MALLO CASDELO
FRANCISCO JAVIER BELLAS BOUZA

FECHA: MARZO DE 2026

Índice

1	Introducción	2
2	Descripción del conjunto de datos	2
2.1	Lectura de los datos	4
2.2	Transformación de los valores del conjunto de datos de texto a número	5
2.3	División del conjunto de datos en datos de entrada y salida	6
2.4	División del conjunto de datos en datos de entrenamiento y prueba	7
2.5	Balanceo de datos	7
2.5.1	Undersampling	8
2.5.2	Oversampling	8
2.6	Normalizado	9
2.7	Análisis de componentes principales (PCA)	10
3	Descripción de las pruebas realizadas y las gráficas obtenidas	12
3.1	Algoritmo de clasificación por vecindad k-nn	13
3.2	Algoritmo de clasificación mediante métodos Kernel (SVM)	21
4	Bibliografía	24

1. Introducción

El presente trabajo tiene el objetivo de analizar el funcionamiento de dos técnicas de aprendizaje supervisado sobre un conjunto de datos etiquetados, de modo que se pueda realizar una caracterización objetiva de su funcionamiento en base a los resultados obtenidos. Para ello, primero se escogerá un conjunto de datos que sea válido para utilizarlo en tareas de clasificación, luego se le aplicarán las distintas técnicas de preprocesado que se consideren oportunas para finalmente, aplicar las técnicas de aprendizaje automático y realizar una comparación de sus resultados.

Para poder realizar este análisis, se implementará un programa en Python utilizando la librería *scikit-learn* para aplicar las técnicas de preprocesado de los datos y de aprendizaje automático. Además, se emplearán otras librerías como *Pandas* o *Numpy*, necesarias para la correcta lectura y manejo de los datos.

2. Descripción del conjunto de datos

La elección del conjunto de datos con el que se vaya a trabajar representa el primer paso de este proyecto. Como el objetivo es emplear conjuntos de datos reales, se han consultado distintas opciones en plataformas web como Kaggle o UCI y finalmente, se ha escogido una base de datos que permite clasificar las setas pertenecientes a la familia *Agaricus* y *Lepiota* como o venenosas o comestibles. [1]

El conjunto de datos seleccionado contiene un total de 8124 muestras. Para cada una de ellas se asocian un total de 22 características, cada una con distintas opciones. Aunque los autores de este conjunto de datos consideran distintas posibilidades dentro de cada característica, existen algunas opciones para las que no existe ninguna muestra, por lo que aparecen tachadas. A continuación se exponen cada una de las 22 variables que conforman la base de datos:

1. Forma del sombrero

b → Campana | c → Cónica | x → Convexa | f → Plana |
k → Con mamelón (protuberancia) | s → Deprimido

2. Superficie del sombrero

f → Fibroso | g → Con surcos | y → Escamoso | s → Liso

3. Color del sombrero

n → Marrón | b → Ante/Crema | c → Canela | g → Gris | r → Verde | p → Rosa |
u → Morado | e → Rojo | w → Blanco | y → Amarillo

4. Hematomas o manchas

t → Sí | f → No

5. Olor

a → Almendra | l → Anís | c → Creosota | y → Pescado | f → Fétido | m → Moho |
n → Inoloro | p → Picante | s → Especiado

6. Inserción de las láminas
a → Unidas | ~~d → Decurrentes~~ | f → Libres | ~~n → Escotadas~~
7. Espaciado de las láminas
c → Cerradas | w → Apiñadas | ~~d → Distantes~~
8. Tamaño de las láminas
b → Anchas | n → Estrechas
9. Color de las láminas
k → Negro | n → Marrón | b → Ante/Crema | h → Chocolate | g → Gris | r → Verde
o → Naranja | p → Rosa | u → Morado | e → Rojo | w → Blanco | y → Amarillo
10. Forma del tallo o pie
e → Ensanchado hacia la base | t → Se estrecha hacia la base
11. Raíz del tallo
b → Bulbosa | c → En forma de maza | ~~u → En forma de copa~~ | e → Cilíndrica |
~~z → Rizomorfos~~ | r → Con raíz | ? → No hay registro
12. Superficie del tallo por encima del anillo
f → Fibrosa | y → Escamosa | k → Sedosa | s → Lisa
13. Superficie del tallo por debajo del anillo
f → Fibrosa | y → Escamosa | k → Sedosa | s → Lisa
14. Color del tallo por encima del anillo
n → Marrón | b → Ante/Crema | c → Canela | g → Gris | o → Naranja | p → Rosa |
e → Rojo | w → Blanco | y → Amarillo
15. Color del tallo por debajo del anillo
n → Marrón | b → Ante/Crema | c → Canela | g → Gris | o → Naranja | p → Rosa |
e → Rojo | w → Blanco | y → Amarillo
16. Tipo de velo
p → Parcial | ~~u → Universal~~
17. Color del velo
n → Marrón | o → Naranja | w → Blanco | y → Amarillo
18. Número de anillos
n → Ninguno | o → Uno | t → Dos
19. Tipo de anillo
~~c → Cortina~~ | e → Sutil | f → Acampanado | l → Grande | n → No hay anillo |
p → Péndulo | ~~s → Envainado~~ | ~~z → En zona~~

20. Color de la esporada

k → Negro | n → Marrón | b → Ante/Crema | h → Chocolate | r → Verde |
o → Naranja | u → Morado | w → Blanco | y → Amarillo

21. Población

a → Abundante | c → Apiñada | n → Numerosa | s → Dispersa | v → Varias | y → Solitaria

22. Hábitat

g → Pastizales | l → Hojarasca | m → Praderas | p → Senderos | u → Urbano | w → Vertederos | d → Bosques

Es necesario aclarar que el archivo .csv que contiene el conjunto de datos, tiene definidas cada una de las opciones de cada atributo con una letra. En consecuencia, se ha expuesto el significado de cada letra, no obstante, el conjunto de datos que se utilizará a partir de ahora, tiene definidas cada una de las opciones de cada atributo con una palabra equivalente a su respectiva letra.

Por último, falta definir la característica objetivo que permite determinar si la seta que se está considerando en cada muestra es comestible o bien es venenosa. El objetivo del presente proyecto es establecer patrones utilizando algoritmos de clasificación, entre esta característica objetivo y las otras 22 características descriptivas, para ser capaz de determinar si una seta es venenosa o comestible si se conoce la forma de su sombrero, su color, su olor, el tamaño de las láminas, ...

2.1. Lectura de los datos

Una vez elegido el conjunto de datos, el siguiente paso es leer su contenido de forma correcta. Cada columna representa una característica y cada fila una muestra o seta que se ha analizado.

El conjunto de datos que se ha elegido tiene algunas muestras que no tienen definidas las 22 características, ya que falta alguna sobre la cual no hay información. Los autores de esta base de datos han identificado con el símbolo "?" aquellos datos que faltan, por lo que lo primero que se ha hecho es asociar al símbolo ? un valor nulo (utilizando la librería *Pandas*).

Luego, para comprobar la calidad del conjunto de datos elegido, se ha comprobado la cantidad de datos que faltan en cada una de las características, es decir, la cantidad de muestras para las que no hay información de cada atributo que se contempla. Como resultado, se observa que hay 2481 muestras para la que no hay información del atributo *Raíz del tallo*.

El hecho de que falten datos para alguno de los atributos dificulta que los algoritmos de clasificación encuentren patrones, por lo que conviene eliminar las muestras que no tengan toda la información. Como resultado, se reduce la cantidad de muestras, obteniendo un conjunto de datos compuesto por 5936 muestras que están caracterizadas por los 23 atributos.

Por otro lado, se observa que la columna *Poisonous* es la que define la característica o atributo objetivo que permite clasificar cada seta en comestible o venenosa. Realizando un recuento de cuantas de las setas analizadas son comestibles o venenosas, se obtiene que 3768 son aptas para el consumo y las 2168 restantes no lo son. Esto refleja que el 63,48 % de las muestras son comestibles, lo que implica que los datos no están balanceados. Esto representa

un problema para los algoritmos de clasificación, ya que provoca que el modelo determine de manera sesgada si una seta es apta o no para el consumo, tendiendo a considerarla como comestible. Este problema se abordará más adelante.

A continuación, se muestra el código que se ha utilizado para realizar el análisis que se ha descrito en el presente apartado.

```
1 df_orig=pd.read_csv('mushrooms1.csv', na_values=["?"]) #  
2     Lectura de los datos y conversión del símbolo "?" a NaN.  
3 df_orig.head()  
4 df_orig.shape #Se consulta la dimensión del conjunto de datos  
5 df_orig.isnull().sum() #Contabilización de la cantidad de  
6     muestras para las que faltan datos en cada atributo o  
7     característica.  
8 df=df_orig.dropna(axis=0) #Se eliminan las filas que contienen  
9     alguna columna que tenga un valor nulo y se guarda todo en  
     la variable "df".  
df.shape  
df.isnull().sum() #Se consulta si existe algún valor nulo.  
df['poisonous'].value_counts() #Agrupar los valores únicos que  
     hay en la columna "Poisonous" y devuelve la cuenta.
```

Listing 1: Eliminar filas con información incompleta

2.2. Transformación de los valores del conjunto de datos de texto a número

Un problema que presenta este conjunto de datos es que todos sus valores están en formato texto, lo que imposibilita realizar todo el estudio sobre este conjunto de datos. Para evitar esto, se pasan todos los valores presentes en el conjunto de datos de texto a número mediante la siguiente técnica de preprocesado:

```
1 encoder = preprocessing.OrdinalEncoder(dtype=int) #Se le  
2     ordena que convierta palabras a números enteros  
3 df_encoded = encoder.fit_transform(df[['poisonous', "cap-  
4     shape","cap-surface","cap-color","bruises","odor",  
5     "gill-attachment","gill-spacing","gill-size","gill-color",  
6     "stalk-shape","stalk-root",  
7     "stalk-surface-above-ring","stalk-surface-below-ring","stalk-  
8     color-above-ring",  
9     "stalk-color-below-ring","veil-type","veil-color","ring-  
     number","ring-type",  
     "spore-print-color","population","habitat"]]) #Conversión de  
     palabras a enteros  
encoder.categories_ #Muestra el diccionario que la  
     herramienta "encoder" ha creado
```

```
10 df_encoded #Se muestran los números enteros que definen el
    conjunto de datos
11
12 #Se guardan en cada una de las columnas de la variable "df"
    todos los datos originales convertidos a número
13
14 df["poisonous"] = df_encoded[:,0]
15 df["cap-shape"] = df_encoded[:,1]
16 df["cap-surface"] = df_encoded[:,2]
17 df["cap-color"] = df_encoded[:,3]
18 df["bruises"] = df_encoded[:,4]
19 df["odor"] = df_encoded[:,5]
20 df["gill-attachment"] = df_encoded[:,6]
21 df["gill-spacing"] = df_encoded[:,7]
22 df["gill-size"] = df_encoded[:,8]
23 df["gill-color"] = df_encoded[:,9]
24 df["stalk-shape"] = df_encoded[:,10]
25 df["stalk-root"] = df_encoded[:,11]
26 df["stalk-surface-above-ring"] = df_encoded[:,12]
27 df["stalk-surface-below-ring"] = df_encoded[:,13]
28 df["stalk-color-above-ring"] = df_encoded[:,14]
29 df["stalk-color-below-ring"] = df_encoded[:,15]
30 df["veil-type"] = df_encoded[:,16]
31 df["veil-color"] = df_encoded[:,17]
32 df["ring-number"] = df_encoded[:,18]
33 df["ring-type"] = df_encoded[:,19]
34 df["spore-print-color"] = df_encoded[:,20]
35 df["population"] = df_encoded[:,21]
36 df["habitat"] = df_encoded[:,22]
```

Listing 2: Transformación de los valores de texto a número

Al ejecutar este código se sustituyen los valores que presenta el conjunto de datos en formato texto a la hora de clasificar los tipos de setas por números enteros, lo que nos permitirá trabajar con estos datos más adelante sin problema.

2.3. División del conjunto de datos en datos de entrada y salida

Una vez que se tiene el conjunto de datos en donde para cada atributo se le asocia un número que define dicha característica, se procederá a definir cuales son los datos de entrada y salida. Para ello, se seleccionan las columnas que se utilizarán como entradas y se convierten en un array de *NumPy*, para poder utilizarlo con los algoritmos de clasificación de la librería *Scikit-learn* (con la columna que se utilizará como salida se hace lo mismo), utilizando el siguiente código:

```
1 feature_df = df[df.columns[1:df.shape[1]]] #Se cogen todas
2     las columnas menos la primera.
3 X = feature_df #Se define el conjunto de datos de entrada
```

```
4 X_readed = np.asarray(feature_df) #Se convierte el conjunto
   de datos de entrada en una matriz NumPy
5 X_readed[0:5]
6 y_readed = np.asarray(df['poisonous']) #Se coge la primera
   columna como dato de salida
7 y_readed[0:5]
```

Listing 3: División del conjunto de datos en datos de entrada y salida

La columna "Poisonous" representa la salida, el valor que se quiere predecir, y el resto de columnas constituyen las entradas, los valores a partir de los cuales se realizará la predicción.

2.4. División del conjunto de datos en datos de entrenamiento y prueba

A la hora de entrenar un modelo, este necesita una cantidad de datos de *entrenamiento* (X_{train} e Y_{train}), los cuales estudiará y tratará de encontrar patrones para que, posteriormente, pueda realizar predicciones sobre unos *datos de prueba* (X_{test} e Y_{test}). Esto permitirá afinar la configuración modelo para luego emplearlo en la realidad.

Por lo tanto, este paso es clave para alcanzar el objetivo de definir un modelo que sea capaz de clasificar si una seta es comestible o venenosa, una vez se tiene la información de los 22 atributos que se están utilizando.

El código empleado para realizar esta separación de los datos es el siguiente:

```
1
2 X_train, X_test, y_train, y_test = train_test_split(X_readed,
   y_readed, random_state = 1)
3 print ('Train set:', X_train.shape, y_train.shape)
4 print ('Test set:', X_test.shape, y_test.shape)
```

Listing 4: División del conjunto de datos en datos de entrenamiento y prueba

En la función "train_test_split" se podría añadir un valor ("test_size") que permite modificar la proporción de la división de los datos según se considere. Por defecto, esta función establece que el 75 % de los datos sea de entrenamiento y el 25 % restante sean datos de prueba.

Dado que el conjunto de datos que se está utilizando es bastante grande, se ha decidido que esta proporción es adecuada, obteniendo finalmente 4452 filas de datos de entrenamiento y 1484 filas de datos de prueba, a falta de terminar con el proceso de preprocesado.

2.5. Balanceo de datos

Tal y como se mencionó con anterioridad, la base de datos elegida para definir un modelo de clasificación está *desbalanceada*. Es decir, la cantidad de valores en la columna *poisonous* que se corresponden con setas comestibles y con setas venenosas no es la misma, siendo su diferencia igual a 1600 muestras, lo que supone que un 63 % de las setas estudiadas son comestibles y un 37 % son venenosas.

Esto puede influir en la capacidad del algoritmo para predecir correctamente la clase minoritaria (en este caso que una seta sea venenosa), ya que al estar menos representada este puede

ignorarla, haciendo que el cálculo de la precisión del modelo no se asemeje completamente a la realidad.

Por lo tanto, para corregir esto, se utiliza la librería *Imbalanced-learn* para realizar el balanceo del conjunto de datos, el cual se puede realizar siguiendo dos filosofías distintas: eliminando muestras de la clase mayoritaria (*Undersampling*), o bien generando datos *sintéticos* de la clase minoritaria (*Oversampling*).

2.5.1. Undersampling

La técnica de *Undersampling* elimina muestras que se clasifican como setas comestibles, es decir, elimina las filas en cuya clase objetivo (que es *Poisonous*) se define una seta como comestible, que es la tipología de muestra mayoritaria. El objetivo es eliminar todas las muestras necesarias para que la cantidad de setas que se clasifican como comestibles y venenosas sea la misma para ambos casos.

Además, el algoritmo *NearMiss* no elimina las muestras de setas comestibles al azar. Calcula matemáticamente cuáles son las setas comestibles que están más cerca (o se parecen más) a las setas venenosas y se queda con esas, descartando las comestibles que son demasiado "obvias". De esta manera, el futuro modelo de clasificación aprenderá a distinguir los casos más difíciles.

El código empleado para realizar el balanceo de datos utilizando la técnica de *Undersampling* es el siguiente:

```
1 unique, counts = np.unique(y_train, return_counts=True)
2 #Escanea la variable "y_train" y contabiliza cuantas muestras
3   hay de cada tipo
4 sm = NearMiss() #Inicializa el algoritmo NearMiss y lo guarda
5   en la variable "sm"
6 X_train, y_train= sm.fit_resample(X_train, y_train) #Se
7   elimina el exceso de muestras de la clase mayoritaria y se
8   sobreescriben las variables X_train e y_train con los datos
9   ya balanceados.
10
11 print('\nBalanceado undersampling:', X_train.shape, y_train.
12       shape)
13 unique, counts = np.unique(y_train, return_counts=True)
14 print(dict(zip(unique, counts))) #Comprobación de que el
15   proceso de balanceado se ha realizado con éxito
```

Listing 5: Balanceo del conjunto de datos: Undersampling

2.5.2. Oversampling

La técnica de *Oversampling* genera muestras que se clasifican como setas venenosas, es decir, identifica cuál es el tipo de muestra minoritaria y añade filas que sean de ese tipo. El objetivo es generar de forma sintética todas las muestras necesarias para que la cantidad de setas que se clasifican como comestibles y venenosas sea la misma para ambos casos.

El algoritmo *SMOTE*, no se limita a copiar y pegar muestras de tipo seta venenosa, sino que genera nuevas muestras muy similares a las ya existentes, de forma que sus propiedades sean realistas para una seta venenosa. Además, definiendo la opción *random_state = 1*, se consigue que siempre genere las mismas muestras cada vez que se ejecuta el código (es lo que se conoce como una *semilla aleatoria*).

El código empleado para realizar el balanceo de datos utilizando la técnica de *Oversampling* es el siguiente:

```
1 unique, counts = np.unique(y_train, return_counts=True) #  
2     Escanea y cuenta cuántas setas hay de cada tipo antes de  
3     hacer la transformación.  
4 sm = SMOTE(random_state=1) #Inicializa el algoritmo SMOTE y  
5     lo guarda en la variable "sm"  
6 X_train, y_train= sm.fit_resample(X_train, y_train) #Generaci  
7     ón de muestras sintéticas del tipo minoritario (setas  
8     venenosas) y se sobrescriben las variables X_train e  
9     y_train con los datos ya balanceados.  
10  
11 print('\nBalanceado oversampling:', X_train.shape, y_train.  
12     shape)  
13 unique, counts = np.unique(y_train, return_counts=True)  
14 print(dict(zip(unique, counts))) #Comprobación de que el  
15     proceso de balanceado se ha realizado con éxito
```

Listing 6: Balanceo del conjunto de datos: Oversampling

2.6. Normalizado

Una vez se ha realizado el balanceo de los datos, se procederá a realizar el escalado del conjunto de datos de entrenamiento y de prueba. Esto es fundamental para evitar que los algoritmos basados en distancias (como k-NN o SVM) tengan un sesgo y le den más peso a unas características que a otras simplemente por tener magnitudes mayores. Para nivelar el rango de todas las variables, se pueden aplicar dos técnicas principales: la Normalización (mediante *MinMaxScaler*, que comprime los datos a un rango de 0 a 1) o la Estandarización (mediante *StandardScaler*, que centra los datos en una media de 0 y desviación estándar de 1).

El método *MinMaxScaler* viene representado por la siguiente fórmula:

$$xi = \frac{xi - \min(xi)}{\max(xi) - \min(xi)}$$

MinMaxScaler se caracteriza por poder transformar una característica para que quede, habitualmente entre un rango fijo de 0 y 1, permitiendo reescalar las características de un conjunto de datos y conservar la forma de la distribución: el porcentaje de variación entre un dato y otro no cambia. Esta técnica difiere un poco con respecto a *StandardScaler*, ya que al dato con mayor valor del rango se le asocia un 1 y al dato con menor valor se le asocia un 0.

El *StandardScaler* tiene como objetivo modificar todos los valores del conjunto de datos de tal forma que tengan un valor de media 0 y un valor de desviación típica de 1, tratando

de aproximar los datos a la distribución normal estándar lo máximo posible. La fórmula que aplica es la siguiente:

$$x_i = \frac{x_i - \mu_i}{\sigma_i}$$

Finalmente, el código que se ha utilizado para el MinMaxScaler es el siguiente:

```
1 scaler = preprocessing.MinMaxScaler()
2
3
4 X_train = scaler.fit_transform(X_train)
5 # aplica los calculos sobre el conjunto de datos de
   entrenamiento para escalarlos
6 X_test = scaler.transform(X_test)
7 # aplica los calculos sobre el conjunto de datos de prueba
   para escalarlos
```

Listing 7: Normalización mediante MinMaxScaler

Y, para el caso de StandarScaler es el siguiente:

```
1 scaler = preprocessing.StandardScaler()
2
3
4 X_train = scaler.fit_transform(X_train)
5 # aplica los calculos sobre el conjunto de datos de
   entrenamiento para escalarlos
6 X_test = scaler.transform(X_test)
7 # aplica los calculos sobre el conjunto de datos de prueba
   para escalarlos
```

Listing 8: Normalización mediante StandarScaler

Realizando pruebas de exactitud y precisión, se pudo observar que ambos casos proporcionan resultados muy similares en los dos ámbitos, aunque definitivamente no funcionan igual, esto se puede observar en las gráficas que se muestran a continuación, que al realizar el escalado de forma distinta varían las densidades de muestras en el conjunto. Además, también se puede observar que si tiene un efecto claro sobre el conjunto de datos donde se equiparan los resultados obtenidos mucho más, haciendo la muestra más homogénea.

2.7. Análisis de componentes principales (PCA)

Una vez se ha llevado a cabo la normalización de los 3 conjuntos de datos (original, original con undersampling y original con oversampling), se procederá a realizar el PCA para buscar conjuntos de ejes de coordenadas en los que algunas de las dimensiones no sean necesarias para estos 3 conjuntos de datos.

Para ello se utilizará el algoritmo *PCA* implementando la librería *scikit-learn*. Inicialmente utilizaremos el algoritmo sin parámetros para analizar el porcentaje de varianza de todos los componentes utilizando *explained_variance_ratio_*, que devuelve una lista en este caso, con 22 elementos, donde cada elemento de la lista indica el porcentaje de variabilidad que representa ese componente. Además, sumando estos porcentajes de variabilidad se consigue saber

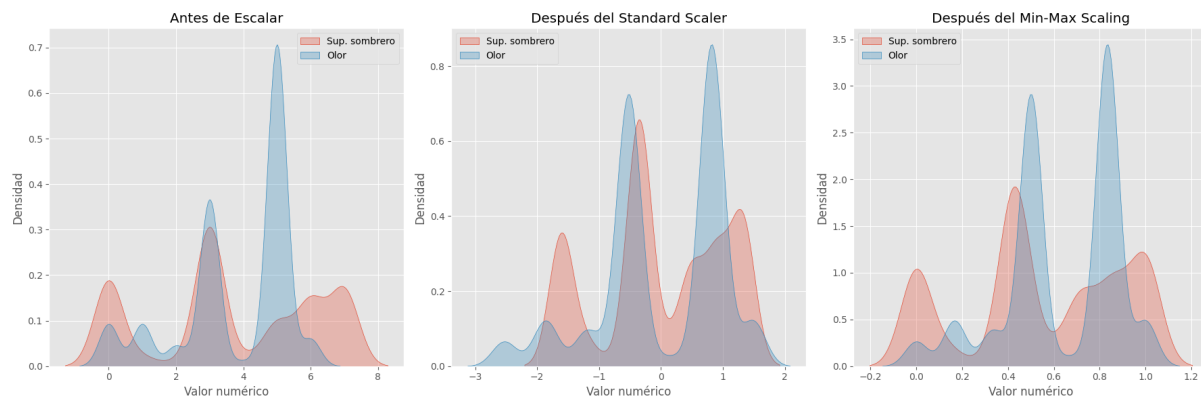


Figura 1: Gráfica de variación de la densidad y el escalado de los datos

la varianza acumulada correspondiente al número de componentes utilizado,s cumpliéndose siempre que la suma de todos los elementos de la lista dará 1 como resultado. El código que se ha utilizado para obtener la varianza y la varianza acumulada es el siguiente:

```

1  mypca = PCA()
2  mypca.fit(X_train)
3
4
5  mypca.explained_variance_ratio_ # muestra todos los
   explained_variance_ratio_
6
7  mypca.explained_variance_ratio_.sum() # Suma todos los
   explained_variance_ratio_
8
9  print("\n Varianza que aporta cada componente:")
10 variance = mypca.explained_variance_ratio_
11 print(variance)
12
13 print("\n Varianza acumulada:")
14 acumvar = variance.cumsum()
15
16 for i in range(len(acumvar)):
17 print(f" {(i+1):2} componentes: {acumvar[i]:.8f} ")

```

Listing 9: Varianza y varianza acumulada de los componentes

Conocidos estos porcentajes de varianza acumulada se pueden representar en una gráfica para poder ver claramente cual es la cantidad de componentes que se debería utilizar para hacer PCA.

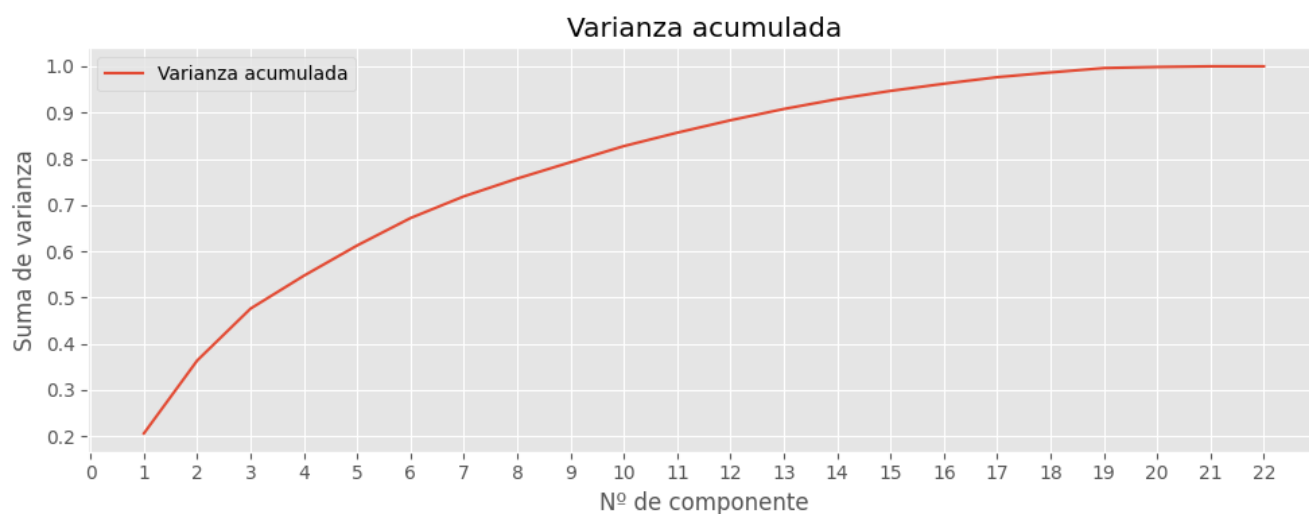


Figura 2: Gráfica de varianza acumulada

Aquí se puede observar como a partir del decimoquinto componente la varianza acumulada tiene un valor muy cercano al 95 %, esto quiere decir que los datos que se van a perder se corresponden con el 5 % de los datos totales. Es por ello que se decide tomar un total de 15 componentes para realizar el cálculo del PCA, ya que con menos componentes las pérdidas de datos se consideran demasiado grandes todavía. Para confirmar que el *Projection loss* no es demasiado alto, se utilizará el siguiente código que nos devolverá su valor:

```
1 mypca15 = PCA(n_components=15)
2 mypca15.fit(X_train)
3 values_proj15 = mypca15.transform(X_train)
4
5 X_projected15 = mypca15.inverse_transform(values_proj15)
6 loss15 = ((X_train - X_projected15) ** 2).mean()
7
8 print("Projection loss (15 components): " + str(loss15))
9
```

Listing 10: Projection loss para 15 componentes

Obteniendo finalmente un valor de *Projection loss* del 5.07 % reduciendo las dimensiones del conjunto de 22 a 15, por lo que se considera aceptable.

3. Descripción de las pruebas realizadas y las gráficas obtenidas

Una vez hemos terminado de realizar todas las técnicas de preprocesado tenemos finalmente 6 posibles conjuntos a probar (original, original + PCA, oversampling sin PCA, oversampling + PCA, undersampling sin PCA, undersampling + PCA). A estos conjuntos de datos se les aplicarán los algoritmos de clasificación que se han seleccionado, en este caso, *k-nn* y *SVM*.

3.1. Algoritmo de clasificación por vecindad k-nn

Este será el primer algoritmo de clasificación que se utilice. A la hora de determinar un dato perteneciente a una clase, se estima que aquellos datos pertenecientes a la misma clase se encontrarán próximos en el espacio de representación, siendo estos los "vecinos" representados en el código por la letra k.

Las pruebas que se realizarán serán para un rango de k de entre 5 y 100, tratando de buscar un valor que otorgue buenos resultados. Además, se intentará que este valor de k no sea ni muy bajo ni muy alto, ya que un valor de k alto da lugar a un modelo muy estable y poco sensible al ruido, pero puede perder detalles. En cambio, un valor de k bajo que proporciona un modelo flexible, que se adapta muy bien a variaciones y detalles, pero que es muy sensible al ruido.

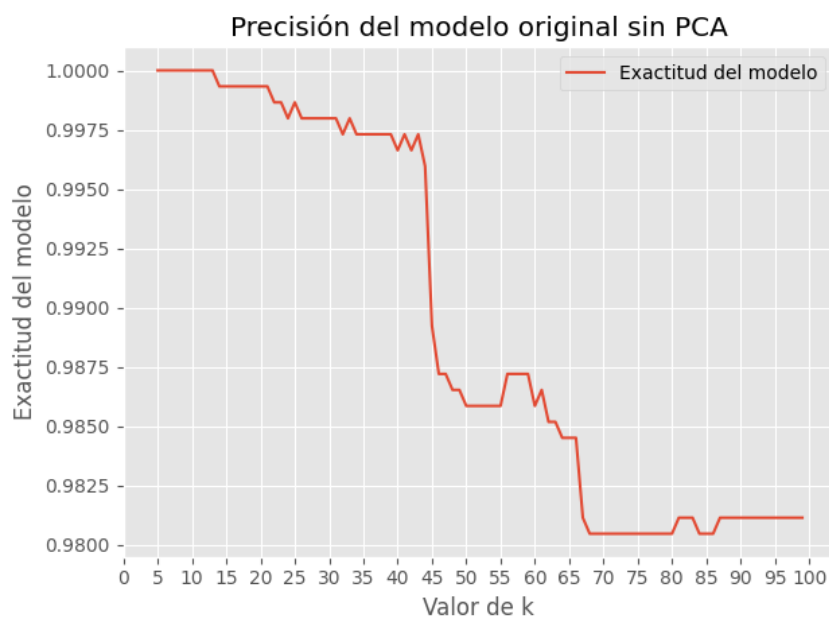
Para realizar pruebas de una forma cómoda y comprobar cual es el mejor caso se ha empleado un bucle *for* que permite obtener el valor de exactitud del modelo y el tiempo de entrenamiento empleado para cada valor de k. De esta manera, se realiza el cálculo de exactitud y tiempo de entrenamiento 5 veces para cada valor de k para, posteriormente, escoger el mejor caso de los 5 calculados.

```
1      Score = np.zeros(95)
2      Posicion = np.zeros(95)
3      Time = np.zeros(95)
4      for i in range(5,100):
5          k = i      # Numero de vecinos
6          Score_ind=np.zeros(5)
7          Posicion_ind=np.zeros(5)
8          Time_ind=np.zeros(5)
9          for u in range (1,6):
10             ini = time.time()
11             # Creamos nuestra instancia de nuestro algoritmo KNN con
12             # K vecinos
13             neigh = KNeighborsClassifier(n_neighbors = k)
14             neigh.fit(X_train,y_train)
15             y_predict_knn = neigh.predict(X_test)
16
17             Time_ind[u-1] = (time.time() - ini)*1000
18             Score_ind[u-1] = neigh.score(X_test, y_test)
19             Posicion_ind[u-1] = i
20             Time[i-5] = np.max(Time_ind)
21             Score[i-5] = np.max(Score_ind)
22             Posicion[i-5] = i
23             print(f"Tiempo entrenamiento para k={i} = {Time[i-5]} ms
24                   ")
25             print("Exactitud media obtenida con k-NN para k={i}: ".
26                   format(i=i), neigh.score(X_test, y_test))
27
28             Valor_maximo = np.max(Score)
29             Tiempo_min = np.min(Time)
30             print(f"El valor maximo de exactitud es de: {Valor_maximo
```

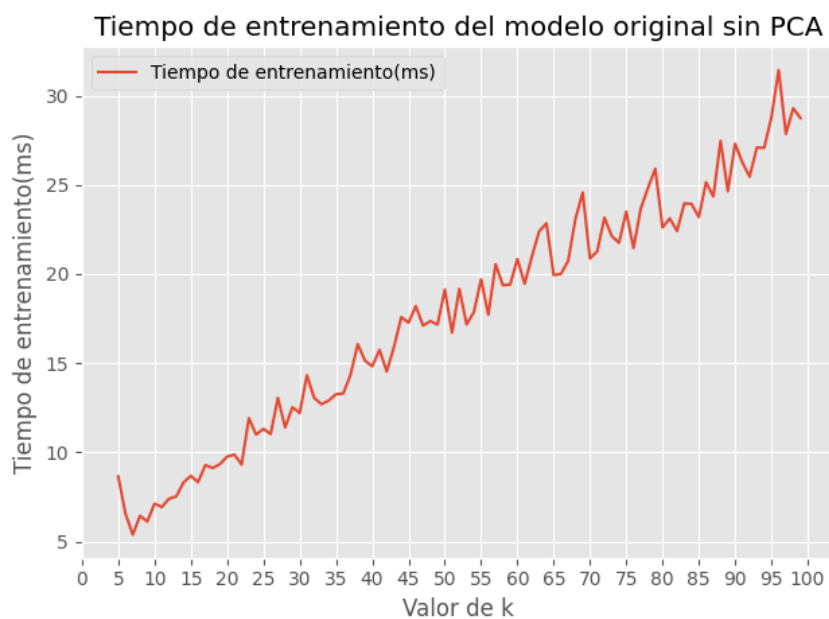
```
    }")
29     print(f"Y esta situado en la posicion: {Score.argmax()+1}
        ")
30     print(f"El valor minimo de tiempo es de: {Tiempo_min}")
31     print(f"Y esta situado en la posicion: {Time.argmin()+1}"
        )
```

Listing 11: Código del algoritmo de clasificación k-nn

Con este código obtendremos de resultado final el caso para el valor máximo de exactitud y el caso para el tiempo mínimo de entrenamiento, para el modelo sin balancear y sin PCA el caso para mayor precisión se da en el rango de entre $k=5$ y $k=13$, mientras que el tiempo de entrenamiento mínimo se dió para $k=7$, pero para poder estudiar más en detalle todos los casos y ver claramente que opción escoger, se mostrarán las gráficas para cada conjunto de datos a estudiar las cuales servirán como guía:



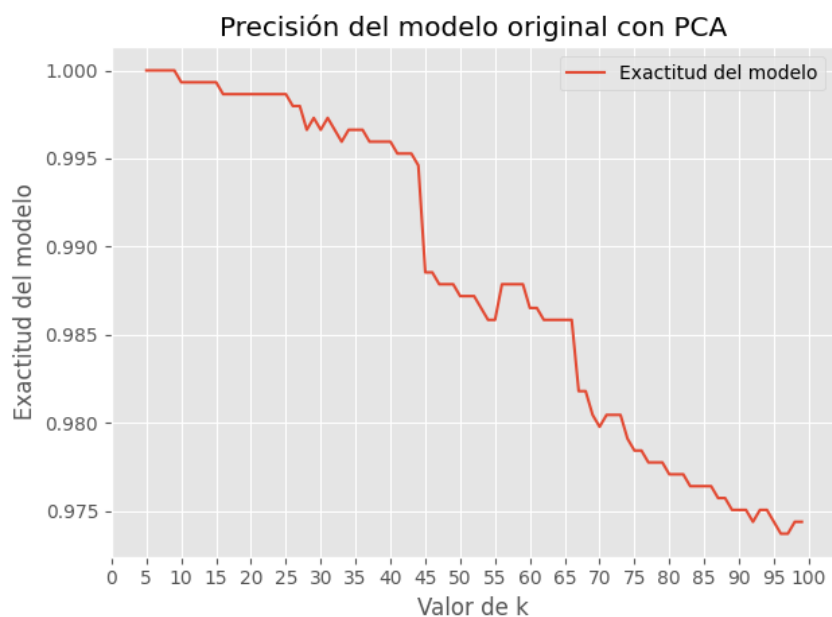
(a) Precisión del modelo sin PCA



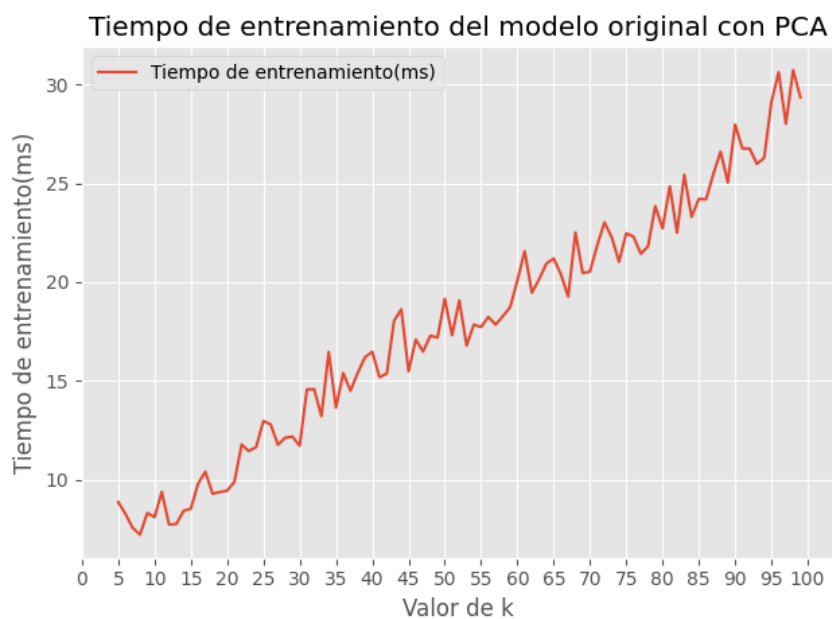
(b) Tiempo de entrenamiento del modelo sin PCA

Figura 3: Precisión y tiempo de entrenamiento del modelo original sin PCA

Por lo tanto nuestro mejor caso será para $k=7$ con una precisión del 100 % y un tiempo de entrenamiento de 5.38 ms. Este proceso se realizará de igual forma para el resto de casos que se realizarán a continuación:



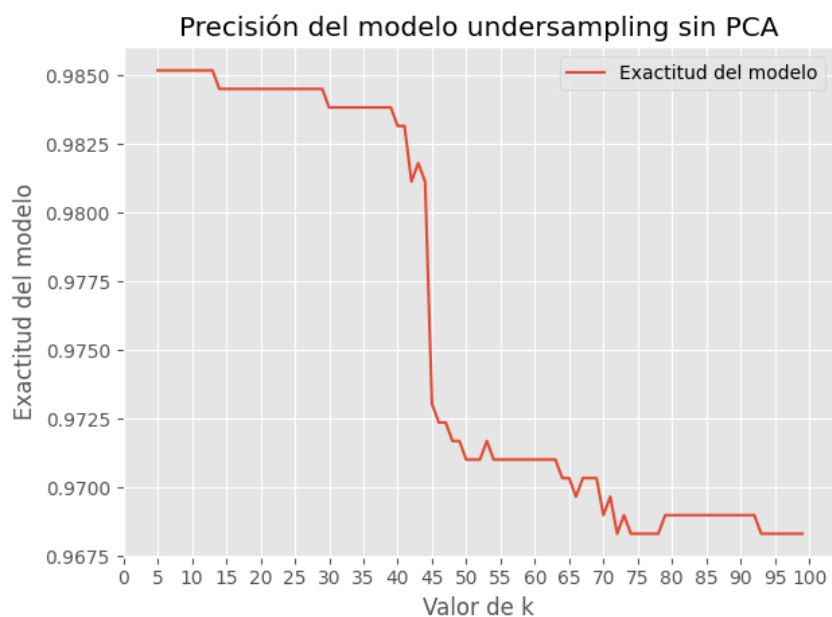
(a) Precisión del modelo con PCA



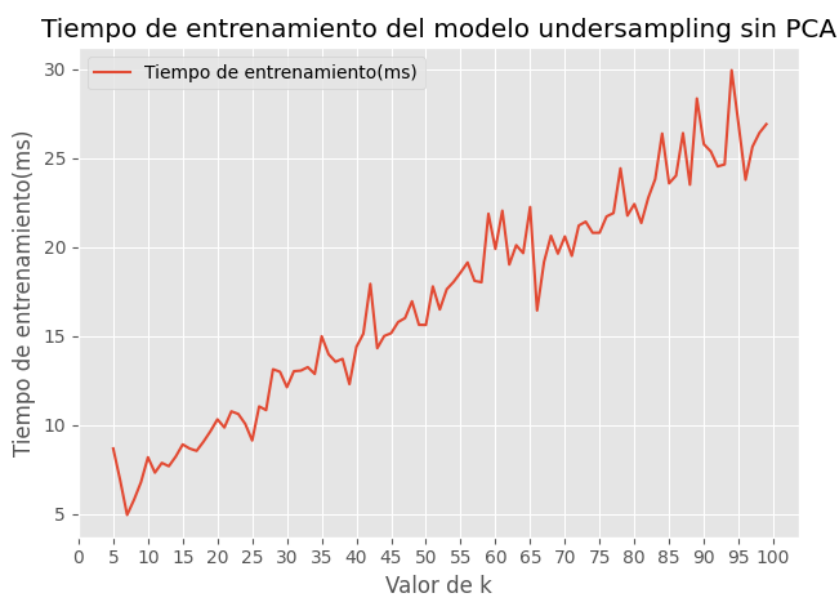
(b) Tiempo de entrenamiento del modelo con PCA

Figura 4: Precisión y tiempo de entrenamiento del modelo original con PCA

En este caso se obtuvo que el mejor caso es para $k=8$ con una precisión del 100 % y un tiempo de entrenamiento de 7.2 ms. Ahora se realizarán las pruebas con este algoritmo para el caso con undersampling:



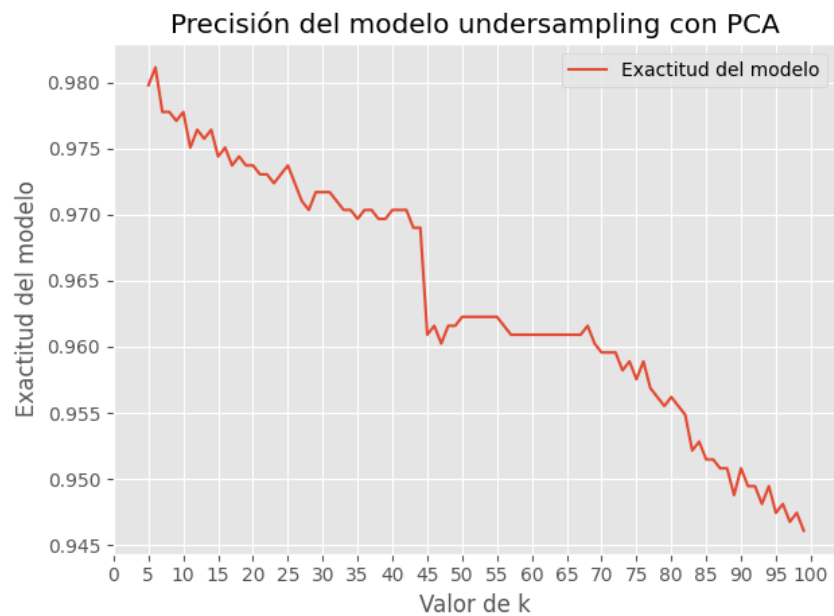
(a) Precisión del modelo sin PCA



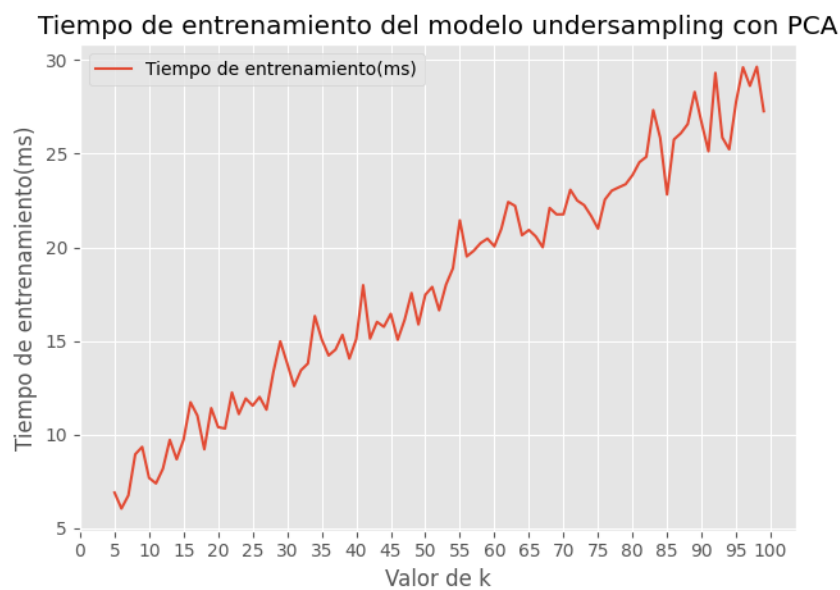
(b) Tiempo de entrenamiento del modelo sin PCA

Figura 5: Precisión y tiempo de entrenamiento del modelo undersampled sin PCA

Tras lo observado en las gráficas y los datos obtenidos con el código, se puede determinar que el mejor caso es para $k=7$ con una exactitud de 0.985 y un tiempo de entrenamiento de 4.96 ms. Aplicando PCA para 15 componentes se obtendrán por lo tanto:



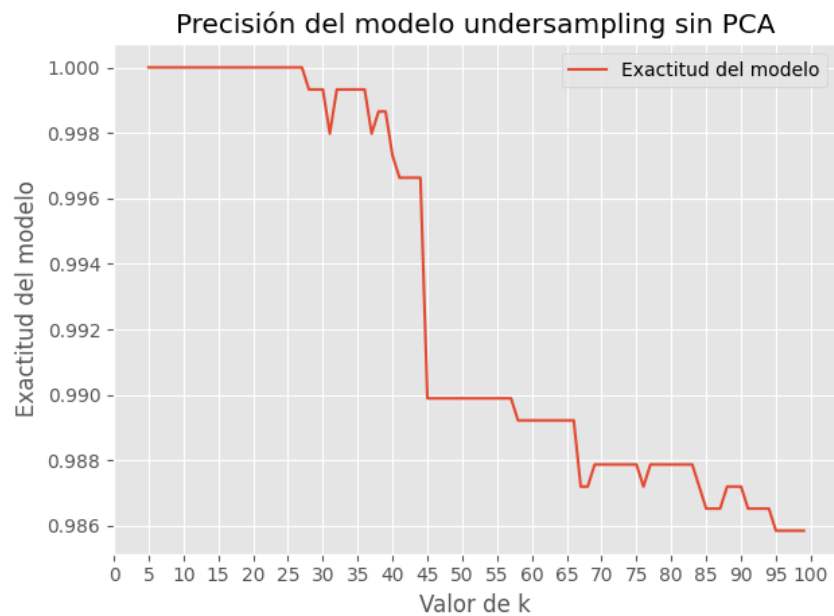
(a) Precisión del modelo con PCA



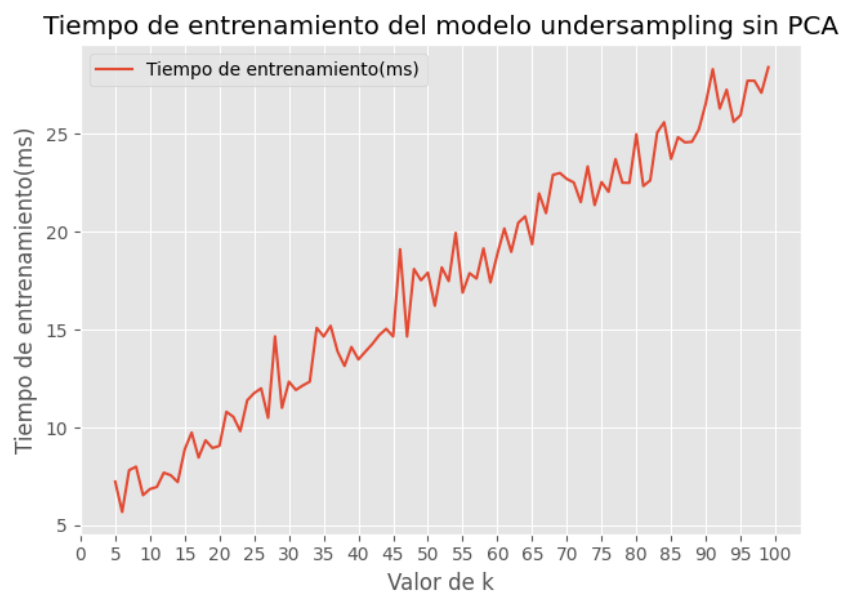
(b) Tiempo de entrenamiento del modelo con PCA

Figura 6: Precisión y tiempo de entrenamiento del modelo undersampled con PCA

En este modelo el mejor caso es para $k=6$ donde se obtiene una precisión de 0.981 y un tiempo de entrenamiento de 6.03 ms. Una vez se han realizado los cálculos para el conjunto original y aplicando undersampling se realizará el mismo proceso para el conjunto de datos al que se le ha aplicado oversampling.



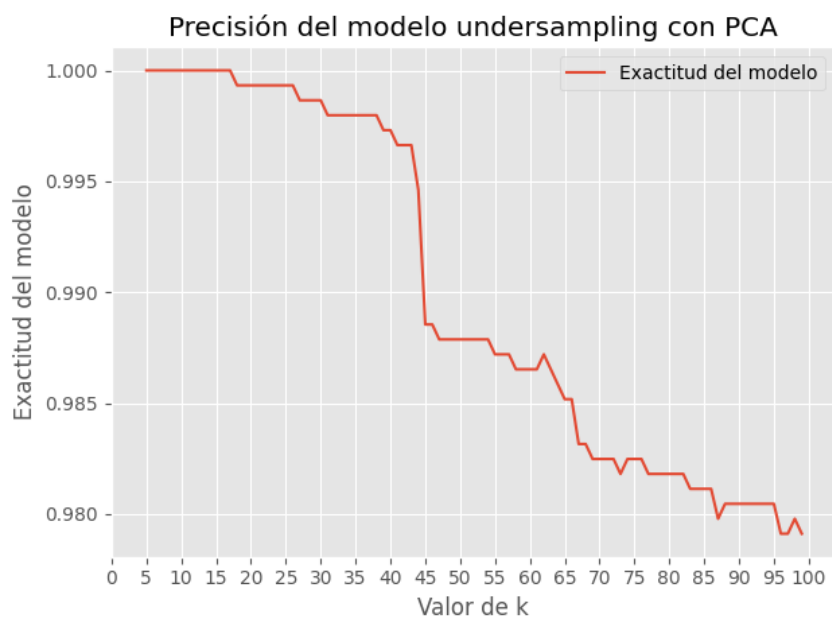
(a) Precisión del modelo sin PCA



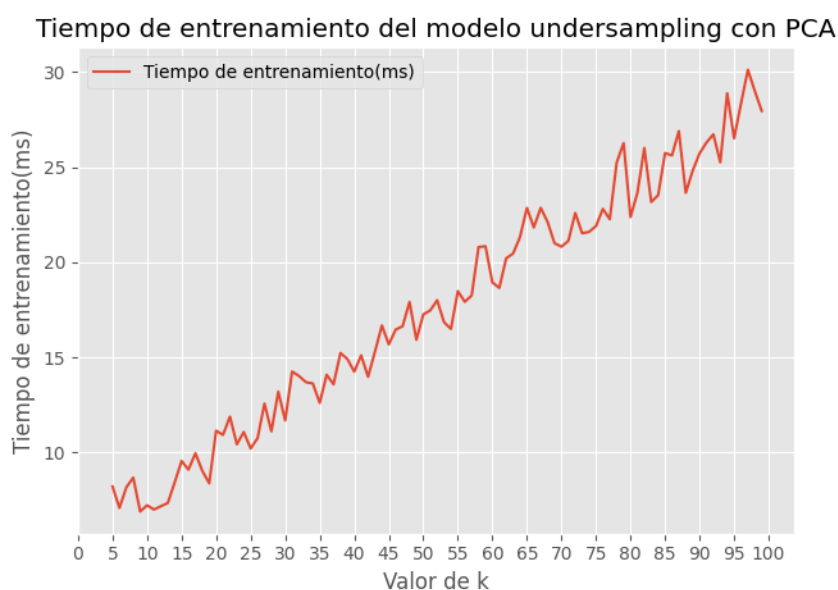
(b) Tiempo de entrenamiento del modelo sin PCA

Figura 7: Precisión y tiempo de entrenamiento del modelo oversampled sin PCA

Donde se puede observar que el mejor caso es para $k=10$, donde conseguimos una precisión del 1.00 con un tiempo de entrenamiento de 6.44 ms. Terminaremos analizando y obteniendo las gráficas para el caso de oversampling con PCA.



(a) Precisión del modelo con PCA



(b) Tiempo de entrenamiento del modelo con PCA

Figura 8: Precisión y tiempo de entrenamiento del modelo oversampled con PCA

Para este último caso, de oversampling con PCA, se ha vuelto a obtener una precisión del 1.00 para valores de k comprendidos entre 5 y 17 por lo que, guiándonos por el tiempo de entrenamiento, se toma que la mejor situación es para $k=8$ donde se obtiene un tiempo de entrenamiento de 6.93 ms, siendo este el mejor caso para oversampling con PCA.

Para poder ver de una forma más clara los resultados finales obtenidos por cada conjunto de datos, se decide realizar una tabla para representarlos:

Conjunto de datos	Exactitud del modelo	Tiempo de entrenamiento	Nº Vecinos
Original sin PCA	1.000	5.38 ms	k=7
Original con PCA	1.000	7.20 ms	k=8
Con undersampling sin PCA	0.985	4.96 ms	k=7
Con undersampling con PCA	0.981	6.03 ms	k=6
Con oversampling sin PCA	1.000	6.44 ms	k=10
Con oversampling con PCA	1.000	6.93 ms	k=8

Tabla 1: Resultados obtenidos para los conjuntos de datos

Observando esta tabla se puede observar que la precisión del conjunto de datos es extremadamente alta, consiguiendo para aquellos casos con mayor cantidad de datos, es decir, el conjunto de datos original y el conjunto de datos con oversampling, que cuentan con una cantidad de filas muy superior al caso de undersampling, consiguen una precisión del 100 %.

Esto nos quiere decir que la cantidad de datos que se tienen para entrenar el modelo es un factor clave a la hora de conseguir precisiones altas con el mismo, aunque esto también esté sujeto a aumentos en el tiempo de entrenamiento, como se puede ver en la comparativa entre los conjuntos de datos de aplicación de undersampling sin PCA y oversampling sin PCA, donde a pesar de que con el oversampling se consigue una precisión del 100 %, las diferencias en los tiempos de entrenamiento es de 1.48 ms. Este valor aunque no implica gran cosa a simple vista, supone un aumento en los tiempos de entrenamiento del 30 % en comparación con el caso con undersampling.

Para la aplicación de PCA se puede observar como, al disminuir el número de dimensiones que se analizan de 22 a 15 disminuye la exactitud del modelo sutilmente, como se puede observar en los 2 casos con undersampling, pero no solo eso, también es notable un aumento en los tiempos de entrenamiento en relación con los casos donde no se aplica lo que nos lleva a una duda considerable con respecto a los beneficios de su aplicación.

Por lo tanto, si nos guiásemos únicamente por la tabla resumen, el mejor caso sería para el conjunto de datos original sin PCA. Esta afirmación tiene ciertos matices ya que este modelo podría tener algún tipo de sesgo a la hora de predecir debido al propio conjunto de datos. Esto se debe a que la correlación entre setas venenosas y comestibles no es 50/50 sino que se tienen 1600 casos adicionales de setas comestibles en comparación con las setas venenosas. Es por esto que finalmente se decide que el modelo óptimo es el que se corresponde con el conjunto de datos con oversampling sin PCA aplicando un valor de k=10, ya que se busca priorizar la exactitud del modelo frente a un tiempo de entrenamiento más bajo. Por último, no se muestran las matrices de confusión en estos casos ya que para una precisión de 1.00 no son muy representativas, no hay ningún error que se pueda observar.

3.2. Algoritmo de clasificación mediante métodos Kernel (SVM)

Ahora se realizarán las pruebas sobre los distintos conjuntos de datos con el algoritmo SVM. El algoritmo SVM se caracteriza por minimizar el riesgo estructural para reducir la posibilidad de cometer errores futuros. Dentro de este algoritmo, este se puede modificar y variar

de dos formas, cambiando el valor de C o cambiando el tipo de kernel.

Para realizar el estudio de este algoritmo se llevarán a cabo 2 pruebas, la primera consistirá en cambiar el valor de C en el rango de 1-10 para ver de que manera varían los resultados obtenidos, y la segunda será cambiando el tipo de kernel utilizado para tratar de obtener los mejores resultados con nuestro código.

El parametro C es el que controla el peso de los terminos de error, es decir, indica cuanto castiga la SVM los errores de clasificación. Esto quiere decir que con un valor de C alto el sistema será más sensible al ruido y tendrá una tendencia a sobreajustar, lo que lo hace menos fiable en otros conjuntos de datos. Sin embargo para un valor de C bajo el modelo tenderá más a ignorar el ruido presente en el conjunto, pero tiene una mayor tendencia a no capturar bien estructuras complejas.

En lo referente al kernel aplicado, se aplicarán 2 tipos de kernel, kernel lineal y kernel no lineal para tratar de observar como afecta al resultado obtenido. Las principales diferencias entre ambos son que un kernel lineal siempre buscará trazar una frontera lineal para separar las clases, mientras que un kernel no lineal permite crear fronteras de decisión no lineales, muy flexibles y capaces de adaptarse a datos complejos. Por lo tanto, se decidió aplicar los kernel *lineal*, *RBF* y *polinómico*.

Como consecuencia de todos los parámetros que se quieren evaluar se diseñó un código muy similar al aplicado para el algoritmo k-nn solo que en este caso el bucle se realizará para unos valores de C de 1.0 hasta 10.0 e inicialmente, se aplicará para un kernel lineal:

```
1      Score_linear = np.zeros(10)
2      Posicion_linear = np.zeros(10)
3      Time_linear = np.zeros(10)
4      for i in range(1,11):
5          Score_ind=np.zeros(5)
6          Posicion_ind=np.zeros(5)
7          Time_ind=np.zeros(5)
8          for u in range (1,6):
9              ini = time.time()
10             clf_svm = svm.SVC(C=i,kernel='linear')
11             clf_svm.fit(X_train, y_train)
12             y_predict_svm = clf_svm.predict(X_test)
13             Score_ind[u-1] = clf_svm.score(X_test, y_test)
14             Posicion_ind[u-1] = i
15             Time_ind[u-1] = (time.time() - ini)*1000
16             Time_linear[i-1] = np.max(Time_ind)
17             Score_linear[i-1] = np.max(Score_ind)
18             Posicion_linear[i-1] = i
19             print(f"Tiempo entrenamiento para C={i} = {Time_linear[i-1]} ms")
20             print("Exactitud media obtenida con SVM para C={i}: ".format(i=i), clf_svm.score(X_test, y_test))
```

```
22     Valor_maximo = np.max(Score_linear)
23     Tiempo_min = np.min(Time_linear)
24
25
26     print(f"El valor máximo de exactitud es de: {Valor_maximo
27           }")
28     print(f"Y está situado en la posición: {Score_linear.
29           argmax()}")
30     print(f"El valor mínimo de tiempo es de: {Tiempo_min}")
31     print(f"Y está situado en la posición: {Time_linear.
           argmin()}")
32
33     clf_svm = svm.SVC(C=Score_linear.argmax(),kernel='linear',
34                       )
```

Listing 12: Código del algoritmo de clasificación SVM con kernel linear

Con este código obtendremos de resultado final el caso para el valor máximo de exactitud y el caso para el tiempo mínimo de entrenamiento, para el modelo sin balancear y sin PCA el caso para mayor precisión se da para $C=10$, mientras que el tiempo de entrenamiento mínimo se dió para $C=7$, por lo que, para poder estudiar más en detalle todos los casos y ver claramente que opción escoger, se mostrarán las gráficas para cada conjunto de datos a estudiar las cuales servirán como guía:

4. Bibliografía

- [1] Mushroom dataset. url = <https://archive.ics.uci.edu/dataset/73/mushroom>. Consultado: 22-03-2026.